Assignment 1 – Summary

GitHub username	ishan-monie
Repository name	advanced-ml-hw1
Link to the code	https://github.com/ishan-monie/advanced-ml-hw1

All code is in this repository, one folder per question (Python 3.12).

How to run the code

1. Install the packages: `pip install -r requirements.txt`. 2. Open a terminal in the folder of the part you want and run the commands below. Every `test_*.py` prints PASS or FAIL for each check, with the evidence.

Part (folder)	Make the graphs	Run the tests
Shuttle (`q2_shuttle`)	`python run_q1.py`	`python test_q1.py`
CartPole (`q3_policy_gradient`)	`python run_cartpole.py`	`python test_cartpole.py`
Pong (`q3_policy_gradient`)	`python plot_pong.py <3 result folders>`	`python test_pong.py`
CartPole baselines (`q4_baselines`)	`python run_cartpole_baselines.py` (20-30 min)	`python test_cartpole_baselines.py`
Pong baseline and extra (`q4_baselines`)	`python plot_pong_baselines.py`	`python test_pong_baseline.py`

Pong training takes many hours, so the finished Pong runs and models are saved in the repository (`results/`); the graphs and tests use them, and the README lists the training commands. Seeds are fixed, but a different number of computer threads can change a run, so the CartPole baseline runs use 1 thread.

What is in the repository

Folder	What it holds
`q2_shuttle/`	Shuttle problem: enumeration, value iteration, policy iteration; the 3 plots (`figures/`); a notebook that shows them one by one.
`q3_policy_gradient/`	Policy gradient without a baseline for CartPole and Pong; graphs in `figures/`, finished Pong runs in `results/`.
`q4_baselines/`	The same with baselines (CartPole: constant and learned; Pong: learned) and an extra convolutional Pong network; graphs in `figures/`, every run in `results/`.
`report/`	This report (`report.pdf`) and the script that builds it. `report_full.pdf` is the long version that lists every check.

Edge cases and tests: Each part has an Edge cases table (the situation, how we handled it, how we checked it) and a Tests table (what we tried, what we expected beforehand, what happened). The tests change a parameter or the problem, compare with hand-worked or exact answers on tiny problems, remove randomness, or check statistically that a method behaves as it should.

Question 2: Value and Policy Iteration (shuttle problem)

Each period we decide whether to dispatch a shuttle (cost 100, takes up to 15 people) or wait (each person left waiting costs 2). Between 1 and 5 people arrive each period. Everything is a cost, so lower is better in every graph.

Assumptions (where the problem left details open)

- State: only the number of people waiting at the start of a period (0 to 200). Order of events: decide, pay, then the new people arrive.
- Dispatching takes up to 15 people (everybody if fewer). The waiting cost of 2 is charged on the people left after the shuttle has gone.
- Full station: above 200 people the extra arrivals are turned away. We checked that this does not change the answer except right near the limit.
- Discount 0.95; the first period is not discounted. After the last of the 500 periods nothing more is paid.
- Method details: value iteration stops when no value changes by more than 1e-10; policy iteration starts from “never dispatch” and keeps its action on exact ties.

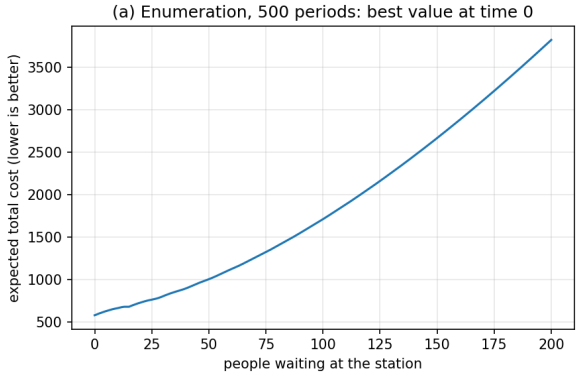


Figure 1 (a). Enumeration, T = 500: best expected cost at time 0 versus people waiting.

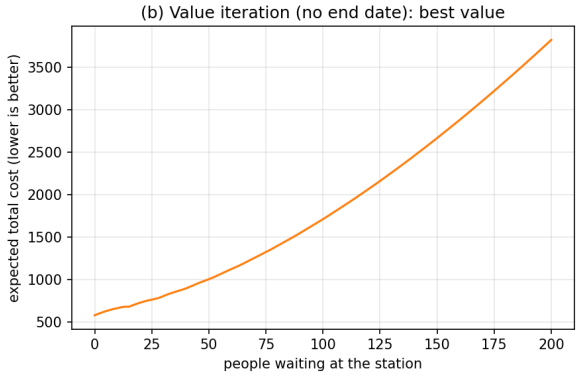


Figure 2 (b). Value iteration, no end date: best expected cost versus people waiting.

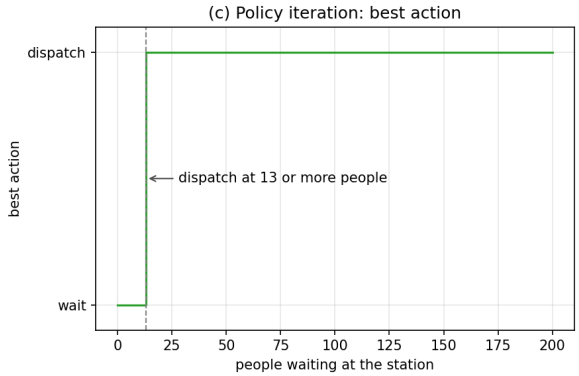


Figure 3 (c). Policy iteration: best action (0 = wait, 1 = dispatch) versus people waiting.

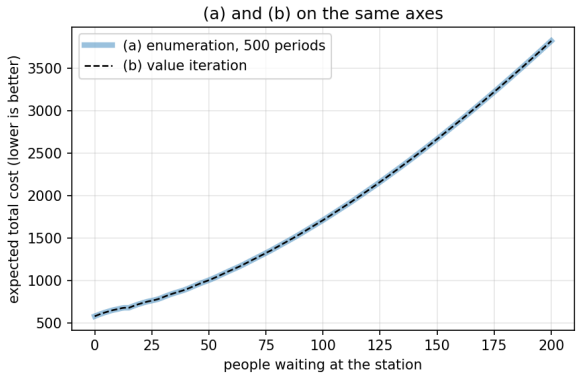


Figure 4. Figures 1 and 2 on the same axes: the curves lie on top of each other.

Enumeration works backwards from the end of the 500 periods. Value iteration repeats the same step with no end date until the numbers stop changing (519 rounds); its curve matches enumeration to about 3 in a billion. Policy iteration settles after 6 rounds on the plan wait with fewer than 13 people, dispatch with 13 or more.

Edge cases

Edge case	How we handled it	How we checked it
Fewer people than a shuttle holds, or an empty station.	Dispatching takes everybody who is waiting (up to 15). With nobody waiting we allow dispatching, but it should never be chosen.	0 people: the best action is to wait. 4 people: dispatch costs 100 and the next state is 1 to 5 people.
Exactly 15 and 16 people: A full shuttle versus one person left behind.	The waiting cost is charged on the people left after the shuttle goes.	15 people: cost 100, next state 1 to 5. 16 people: cost 102, next state 2 to 6.
A full station: The 200-person limit, and the people who would arrive above it.	Extra arrivals are turned away; the transition table is built with the limit built in.	From 198: 20% go to 199 and 80% to 200. With a limit of 300 the best actions are the same (largest cost difference 9.4e-11).
Ties and rounding noise: two actions with the same cost can make policy iteration loop or value iteration flip-flop.	An action is changed only if the new one is better by more than 1e-9, and dispatch is chosen only if it is strictly cheaper.	Policy iteration started from “always dispatch” still found the same best plan; a problem where every action ties stopped after 1 round.
Stopping too early or solving inexactly: value iteration cut short, policy iteration approximated.	Value iteration stops only when no value changes by more than 1e-10; policy iteration solves its equations exactly.	Value iteration from all 0, all 1000 and random starting values agreed to 4e-9; the exact solve matched a slow repeat-until-settled calculation to 9e-13.
Methods that never settle: value iteration that cannot converge; policy iteration that cycles on exact ties.	A limit on the number of rounds (it stops with an error); policy iteration keeps its action on ties.	Forced to fail, both stop with an error. A problem where every action ties stops after 1 round.

Tests

Test	What we expected	What happened
All three methods agree, including policy iteration started from “always dispatch” and value iteration started from 0, from 1000 and from random numbers.	Same costs and same plan every time.	Biggest gap 3.7e-9; the same plan every time.
Hand-worked short horizon: with 1 period left we never dispatch and pay 2 per person; with 2 periods left the cost is 3.9 x people + 5.7.	The program matches the hand calculation.	It matched.
Change the shuttle size to 5, 15 and 30.	A bigger shuttle should wait for more people.	Dispatch at 5, 13 and 17 people.
Extreme prices: free shuttle, free waiting, and a shuttle costing 1,000,000.	Always dispatch when someone waits; never dispatch; never dispatch.	All three as predicted.
Nobody ever arrives, 10 waiting.	Dispatch once (100), not wait forever (400).	The program says 100.0.

Test	What we expected	What happened
Heavy arrivals (16 to 20 per period, more than a shuttle carries).	Our first prediction was wrong: we expected to dispatch from 15 people. Each person left costs 2/(1-0.95) = 40 forever, so dispatching (100) already pays from 3 people.	The program dispatches from 3 up to 152, which the reasoning above explains. It stops at 152 because of the 200 limit (252 with a limit of 300).
Is 500 periods long enough? Compare horizons of 100, 200 and 500 with no end date.	The gap should shrink to nothing.	4.0, then 0.024, then 3e-9.
Simulate the station 3,000 times with the best plan.	The average cost should match the calculated cost.	Simulated 575.8 (± 1.7), calculated 576.7.

All 32 checks (including smaller ones) are in test_q1.py.

With 3 more months: shuttle problem

Everything so far checks that the program solves our model. With more time we would ask whether the model fits a real station: use real arrival data to see if “dispatch at 13” still holds, add rush hours and several shuttles, and see how far the plan moves if the costs of 100 and 2 are really 50% to 150% of what we assumed. We would also test the methods on hundreds of random problems and against a brute-force search on small ones.

Question 3, Part 1: Policy Gradient without a baseline

A small neural network gives the probability of each action. We play whole games choosing actions at random from those probabilities, then make actions that were followed by more (discounted) reward more likely, using Adam. This is the basic version with no baseline.

(a) CartPole-v1

- Network: the 4 state numbers go through one hidden layer of 128 units to 2 action probabilities. Discount 0.95, Adam with the assignment's learning rate 0.01 (we also tried 0.003 and 0.001; the last-100-game averages for 0.01, 0.003 and 0.001 were 285, 249 and 234 with no clear winner, so we kept 0.01).
- Training: 1,500 games, one update per game, actions chosen at random in training and in the 500 test games (a different random seed). Credit: each push gets the discounted reward from that step on. The main model is seed 0, chosen in advance; seeds 1 and 2 show the effect of luck.

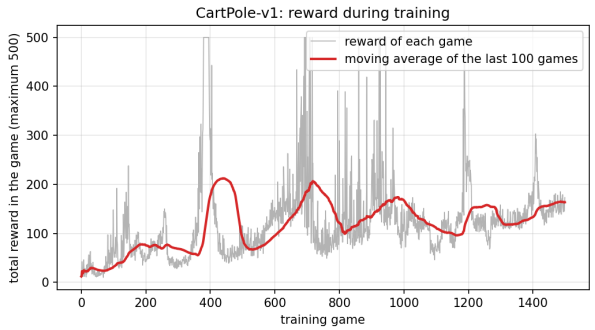


Figure 5. Reward of each training game and the 100-game moving average (seed 0).

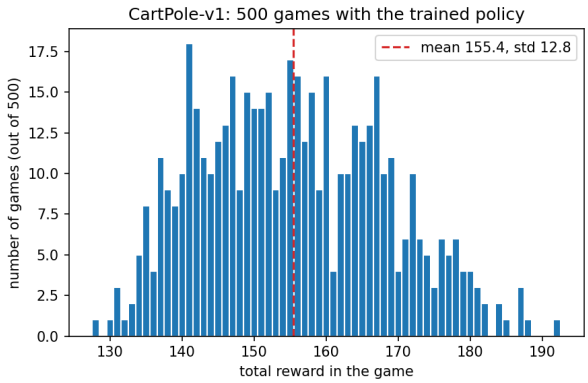


Figure 6. 500 test games with the trained model: mean 155.4, standard deviation 12.8.

The network clearly learns (games last about 20 steps at first and often 200 or more later) but unsteadily, which is what a baseline is meant to fix. On the 500 test games the three seeds scored 155.4 (std 12.8), 500.0 (std 0.0) and 277.5 (std 10.8), so one run says little about the method.

Edge cases

Edge case	How we handled it	How we checked it
The pole falls at once (a game of one step), and the 500-step limit.	A one-step game still gives a valid update. We use CartPole-v1, whose limit is 500, not v0, whose limit is 200.	A one-step game changes the network with finite numbers; every score is between 1 and 500 (the 500 test games ranged from 128 to 192).
Credits growing without limit in a long game.	The discount must be below 1, so credits are capped: with 0.95 no credit can exceed 1/(1-0.95) = 20.	The largest credit in a 500-step game was 20.000.
Test games changing the network by accident.	Test games run with learning switched off, and the network is not updated during them.	Weights were identical before and after the test games, and no learning information was kept.
Log of a probability of 0, and huge numbers.	Probabilities are built from raw scores (logits), so they are never exactly 0.	An extreme network (weights x 10,000) and extreme inputs still gave finite log-probabilities.

Edge case	How we handled it	How we checked it
Results that cannot be repeated.	Seeds are fixed, and the test games use a different seed from training.	The same seed gave an identical training curve; a different seed a different one.

Tests

Test	What we expected	What happened
Hand-worked credits: rewards 1, 1, 1 with discount 0.5.	Credits 1.75, 1.5, 1.0, and a reward never credits earlier pushes.	As expected.
A made-up problem with a known answer: one situation where action 1 pays 1 and action 0 pays 0. The exact gradient of the expected reward is known.	Our learning signal points the same way and has the same size.	Direction match 1.0000, size within 0.9%.
Can that test fail? Repeat it with the sign flipped, and with a learner that ignores the reward.	Both should be caught.	Direction match –1.00 for both.
Tiny learning rate (1e-6).	It should hardly learn and stay near random play (about 22).	The last 100 games averaged 20.2.
Trained against untrained over 500 games each.	Far better (statistical test, t above 10).	155.4 against 22.0, t = 176.
Remove the randomness (always take the most likely action).	About as good as, or better than, random choice.	155.9 against 155.4.

All 24 checks are in test_cartpole.py.

(b) Pong-v5

- Game: we play the right-hand paddle with ALE/Pong-v5; a point gives +1 or –1 and a game ends at 21, so scores run from –21 to +21 (random play scores about –20). Only actions 2 and 3 (up and down) are used. The game is partly random: it repeats our previous push with 25% chance.
- Input: each screen is cleaned with the given `preprocess` (on a copy) into 80 x 80 numbers; the network sees the difference from the previous screen, so it can see motion (the first screen of a game has nothing before it, so it is all zeros). Network: 6,400 inputs, 256 hidden units, 1 output (chance of pushing up). Discount 0.99.
- Credit: the running total restarts at every point scored, so a push is credited only for its own point (a common choice for Pong, and ours). Learning: Adam; the loss is averaged over 4 games per update.
- Learning rate: the assignment's 0.01 and a smaller 0.001 were tried. Rule set before the results: the main model is the one with the higher last-100-game average. Length: Pong is slow, so runs got a time limit instead of a number of games: the 0.001 runs had 6 hours on one GPU card of the course server (6,993 and 6,534 games), the 0.01 run 4 hours on a laptop (9,486 games). An earlier pair of 0.001 laptop runs was cut short when the laptop slept and was replaced by these longer runs.

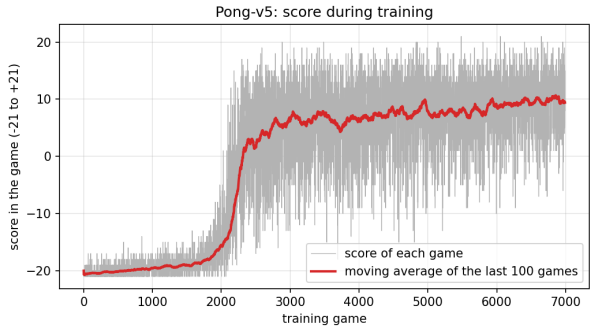


Figure 7. Score of each training game and the 100-game moving average (main model, 6,993 games).

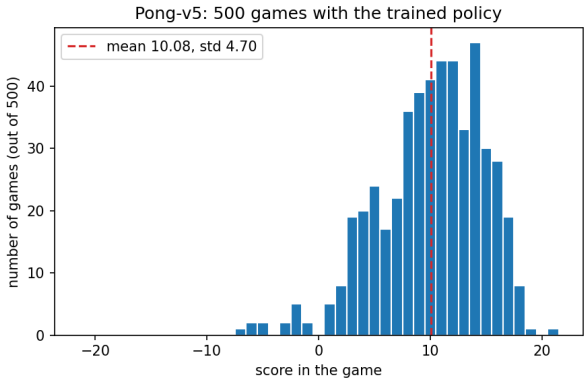


Figure 8. 500 test games with the trained main model: mean 10.08, standard deviation 4.70.

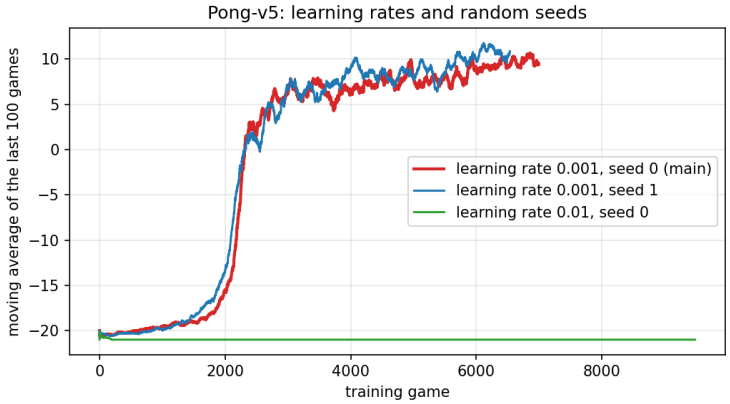


Figure 9. All three runs: learning rate 0.001 with two seeds, and the assignment's 0.01.

The main model stays near –21 for about 2,000 games, then takes off and levels out around +8 to +10. On the 500 test games it scored a mean of 10.08 (standard deviation 4.70); the second seed scored 10.48 (4.98). The assignment's learning rate of 0.01 did not work: it stayed at –21 for all 9,486 games (test mean -21.00), which is why we used 0.001. Test 5 below shows why.

Edge cases

Edge case	How we handled it	How we checked it
The first screen of a game has no screen before it.	The network is shown all zeros for that step.	The first input was all zeros; later inputs showed both +1 and –1 (motion).
Wrong buttons: The game has 6 actions; we use only 2 and 3.	Only up (2) and down (3) can be chosen.	The game confirms that buttons 2 and 3 are RIGHT and LEFT.
Test games changing the network by accident.	Test games run with learning switched off.	Weights were identical before and after test games, and the same seed gave identical test scores.
Credit leaking across points (a loss blamed on pushes from an earlier point).	The running total restarts at every point scored.	See test 1: hand-worked credits, and what would happen without the restart.
NaN or infinite numbers in the screens or rewards, and an empty game.	The update is skipped before the network is used.	Weights were identical afterwards. Writing this test found a crash with NaN screens in our first version; it was fixed.

Edge case	How we handled it	How we checked it
Long games and the time limit: A game takes about 1,000 steps, and a run can stop in the middle of a batch.	Games are kept as light as possible; games already played when the limit hits are still learned from.	One game used about 24 MB, so a batch of 4 is easy to hold; a run stopped after 1 game of a batch of 4 still changed the network.
An extreme network and results that cannot be repeated.	Probabilities are built from raw scores; seeds are fixed.	Finite probabilities with weights x 10,000; the same seed gave an identical training curve.

Tests

Test	What we expected	What happened
1. Hand-worked credits at points: a point won, then one lost (rewards 0, 0, 1, 0, –1; discount 0.9).	0.81, 0.9, 1, –0.9, –1. Without the restart the earlier pushes would wrongly share the later loss.	As expected; without the restart the first credit would fall from 0.81 to 0.15.
2. A made-up problem with a known answer (one screen where pushing up pays 12 and down pays 10), and the same test with the sign flipped.	Our signal matches the exact gradient; the flipped version is caught.	Direction match 1.0000 (size within 1.8%); flipped: –1.00.
3. Trained against untrained over 30 fresh games each.	Far better.	8.6 against –20.2, t = 28.
4. Remove the randomness (always the most likely push).	About as good as, or better than, random choice.	11.2 against 8.6.
5. Why did 0.01 fail? Measure how certain each network is of its push.	The failed network should be almost always certain, so it stops trying other pushes.	99% of its pushes were above 99% certain (and 89% were up), against 70% for the main model; its score was –21.

All 29 checks are in test_pong.py.

With 3 more months: CartPole and Pong

Our results depend a lot on luck (CartPole seeds ranged from 155 to 500; shorter Pong runs of two seeds ended far apart). We would run 20 or more seeds per setting with confidence ranges, train Pong for much longer, switch off each of our own choices in turn (credit restart, frame differences, 4 games per update), search learning rate, network size and batch size, and find out why 0.01 froze. We would also test against a faster opponent and tune the convolutional network.

Question 3, Part 2: Policy Gradient with a baseline

What a baseline is: In Part 1 a push is credited with all the reward that followed it, which is noisy: a push can look good just because the whole stretch went well. A baseline is a reference number that we subtract first: credit = reward that followed – baseline. A push now counts as good only if things went better than the baseline. As long as the baseline does not depend on the push being judged, learning still points the same way on average, but each learning step is less noisy.

Which baselines we chose and how they are implemented:

- Constant baseline (CartPole): one number for every situation, a running average of the rewards from earlier games (about the last 20; 0 before the first game). Earlier games only, so it cannot depend on the current pushes.
- Learned baseline (CartPole and Pong): a second network of the same size as the policy (CartPole 4-128-1; Pong 6,400-256-1) that guesses the reward to expect from each situation. After every policy step it is trained, with its own Adam at the same learning rate, to match the rewards actually earned (squared error). When used as a baseline its output carries no learning information into the policy.
- Everything else is as in Part 1 (network, discount, learning rate, games, test games), so differences come from the baseline. “No baseline” here is exactly Part 1 (a test checks this). Because single runs are so lucky or unlucky, CartPole uses 10 seeds per method (our first try with 3 seeds gave the opposite picture), and Pong 2 seeds. Main models: learned baseline, seed 0, chosen in advance.

CartPole-v1 with baselines

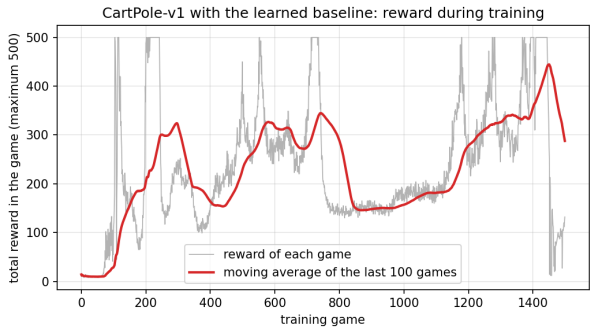


Figure 10. Learned baseline, seed 0: reward of each game and 100-game moving average.

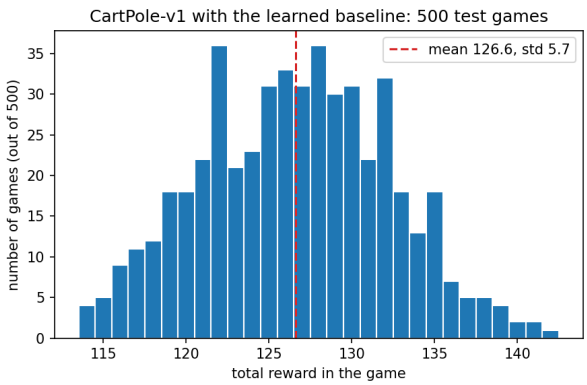


Figure 11. 500 test games, learned baseline, seed 0: mean 126.6, standard deviation 5.7.

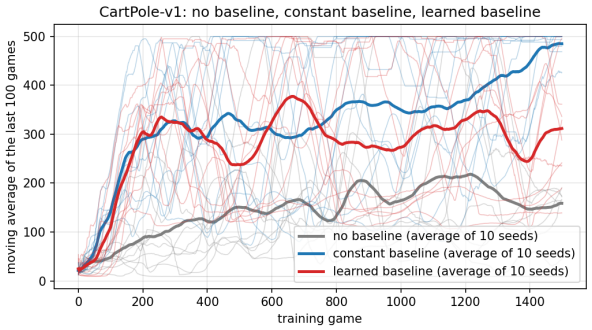


Figure 12. Moving average of the last 100 games: faint lines are seeds, bold lines the average of 10 seeds.

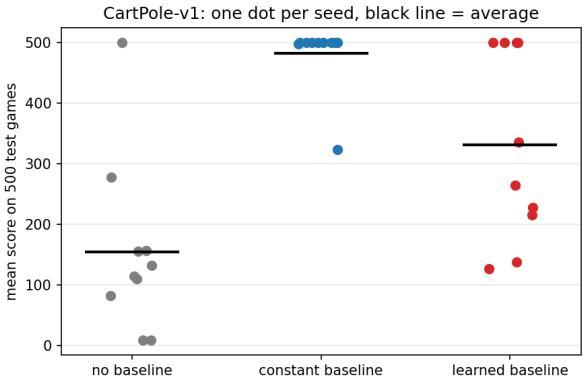


Figure 13. Test score of every seed (500 games each); the black line is the average.

Method	Average test score (10 seeds)	Worst and best seed	Seeds at the 500 limit
No baseline	154.9	9 to 500	1 of 10
Constant baseline	482.1	323 to 500	8 of 10
Learned baseline	330.9	127 to 500	4 of 10

Both baselines help: the constant one scored 482 against 155 with none ($p < 0.001$) and the learned one 331 ($p = 0.018$). The main model (learned, seed 0) scored only 126.6, below its method's average (it fell back in the last 50 training games, Figure 10); we report it because we chose it in advance. We expected the learned baseline to beat the constant one, but the constant one scored higher ($p = 0.015$ on one test, 0.063 on another: weak evidence), and we don't know why yet. All CartPole runs use 1 computer thread (another thread count can change a run).

Pong-v5 with the learned baseline

The baseline network sees the same 6,400 numbers as the policy and is trained to guess the (point-restarting) credits. Runs were asked to play 6,500 games with a 9-hour limit on one GPU card; seed 1 stopped at 5,812 games and the convolutional run at 4,847. Runs are compared on equal numbers of games.

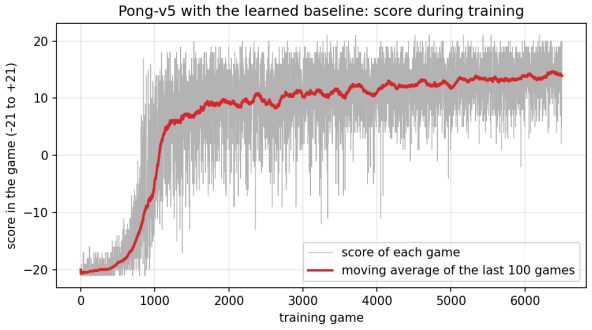


Figure 14. Learned baseline, seed 0: score of each game and the 100-game moving average (6,500 games).

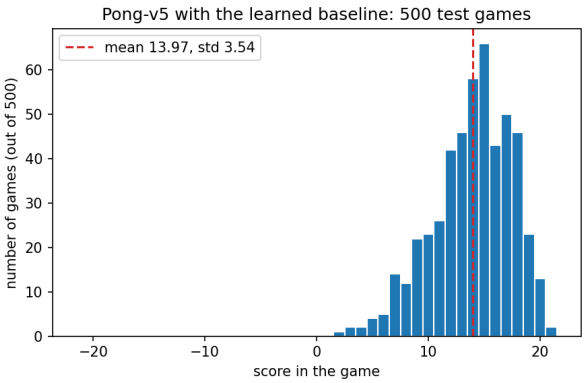


Figure 15. 500 test games: mean 13.97, standard deviation 3.54.

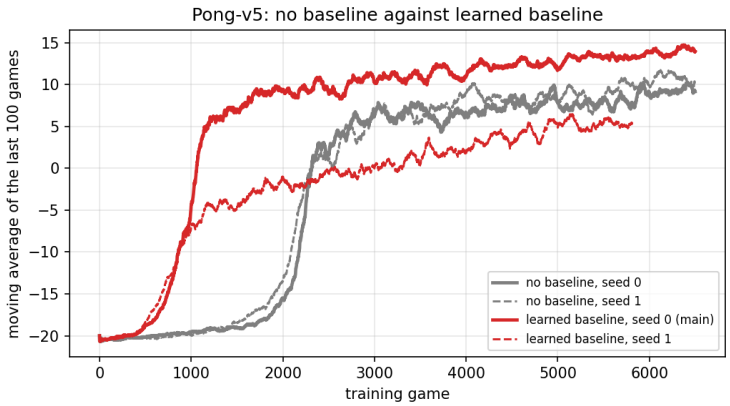


Figure 16. Moving average of the last 100 games with and without the learned baseline (two seeds each).

Run	Games	Average of last 500 games (up to game 6,500)	Game where the average first reached –10	500 test games: mean ± std
No baseline, seed 0	6,993	9.26	2,176	10.08 ± 4.70
No baseline, seed 1	6,534	10.85	2,091	10.48 ± 4.98
Learned baseline, seed 0 (main)	6,500	13.97	862	13.97 ± 3.54
Learned baseline, seed 1	5,812	5.29	870	5.57 ± 5.18

The baseline makes learning start about 2.5 times sooner (the average first reached –10 at game 862 and 870, against 2,176 and 2,091), for both seeds. Final scores are mixed: the main model is the best player we trained (13.97 against 10.08 for the same seed without a baseline, very unlikely to be luck), but seed 1 ended lower (5.57 against 10.48), with fewer games. With two seeds we can say the baseline speeds up the start of learning, not that it always gives a better final player.

As an extra, we also tried a network whose first layer is small picture filters (so a ball is recognised wherever it is), with the learned baseline (one seed, untuned, 4,847 games). It took off earlier than no baseline at all but then levelled off near zero: test score 0.57 ± 5.46, against 13.97 for the plain network. It did not help here, but with one seed and no tuning this says little about convolutional networks in general.

Edge cases

Edge case	How we handled it	How we checked it
A baseline that depends on the push it judges would bend the direction of learning.	The constant baseline uses earlier games only; the learned baseline carries no learning information into the policy's step.	A check on the learned baseline's use found none; see the tests for a baseline that peeks, which is caught.
The first games: nothing to average yet for the constant baseline.	It starts at 0, so the first update equals Part 1's.	The first update was identical to "no baseline".
The new code accidentally changing Part 1.	"No baseline" subtracts a reference of exactly 0, so it is Part 1 under another name.	One learning step gave identical weights, and 60 games of CartPole training gave identical curves.
A bad number reaching the baseline network (NaN or infinite rewards, or NaN inside a game's states).	The whole update is skipped, so neither network changes.	Both kinds of bad input left every weight of both networks identical.

Edge case	How we handled it	How we checked it
The convolutional extra: one screen or a batch of screens, extreme inputs.	The network reshapes any input to pictures and returns one probability per screen.	Both shapes worked; weights x 30 and inputs of hundreds of thousands gave finite numbers; the picture filters themselves received learning.
One lucky or unlucky run deciding the answer, and computer threads.	10 seeds per method on CartPole; 1 thread per run.	Our first try with 3 seeds gave the opposite answer; the same seed with a different thread count gave a different run.

Tests

Test	What we expected	What happened
“No baseline” is exactly Part 1: one learning step, and 60 games of CartPole training.	Identical weights and identical training curves.	Identical in both.
A baseline must not bend where learning points (made-up problem with a known answer, as in Part 1): no baseline, constant, learned.	All match the exact gradient.	Direction match 1.0000 for all; size error 0.092 (none) and 0.001 (baselines) on CartPole.
Can that test fail? A baseline that peeks at the push’s own reward, and one that subtracts twice as much.	Both caught.	Direction match 0.00 and −1.00.
Does a baseline cut the noise? Repeat small batches of 20 pushes and measure how much the learning signal varies.	Much less noise with a baseline.	CartPole 4.95 down to 1.42; Pong 8.44 down to 3.18.
Does the learned baseline learn to expect the right reward? (Made-up problem: expected reward 11.)	A guess near 11.	11.00 (CartPole network), 11.11 (Pong network).
Do baselines help in real training? CartPole, 10 seeds per method; statistical tests of each baseline against none.	Both better.	$p = 0.018$ (learned) and below 0.001 (constant).
Does the Pong baseline make learning start sooner?	Reaches an average of −10 in under 60% of the games.	Game 862 against 2,176; game 870 against 2,091.

All 22 and 26 checks are in test_cartpole_baselines.py and test_pong_baseline.py.

With 3 more months: baselines

We would explain our surprises (the constant baseline beating the learned one on CartPole; one Pong seed climbing more slowly after a fast start), for example by giving the baseline network more learning steps per game or a lower learning rate. We would measure the noise of the learning step during real training, not just on a made-up problem, and compare other baselines (batch average, rescaled credits, a baseline sharing layers with the policy).